

Central Communication and Intelligent Reminder Platform

TECHNICAL SPECIFICATIONS DOCUMENT

Group 6 - Software Engineering Project
Indian Institute of Technology Hyderabad

CS23BTECH11007	Arnav Maiti
CS23BTECH11009	Bhumin Hirpara
CS23BTECH11023	Karan Gupta
CS23BTECH11048	Pranjal Prajapati
CS23BTECH11052	Roshan Y Singh
CS23BTECH11060	Sujal Meshram

Contents

1	Introduction	4
1.1	Purpose	4
1.2	Scope	4
1.3	Definitions and Acronyms	4
2	System Architecture	4
2.1	High-Level Overview	4
2.2	Backend Module Layout	5
2.3	Frontend Module Layout	5
2.4	Data Flow	6
3	Authentication and Security	6
3.1	JWT Token Flow	6
3.2	Request Authorization	6
3.3	Tracking Token Security	7
3.4	Frontend Auth	7
4	Data Model	7
4.1	MongoDB Collections	7
4.1.1	users	7
4.1.2	recipients	7
4.1.3	campaigns	8
4.1.4	templates	8
4.1.5	campaign_dispatch_queue	9
4.1.6	Other Collections	9
5	Campaign Dispatch Pipeline	9
5.1	Overview	9
5.2	Campaign Lifecycle	9
5.3	Priority Scoring Algorithm	10
5.4	Scheduler	10
5.5	Retry Mechanism	10
5.6	Multi-Channel Rendering	11
6	Engagement Tracking	11
6.1	Tracking Endpoints (Public, No Auth)	11
6.2	Event Recording	11
6.3	Engagement Score Updates	11
7	AI-Powered Features	12
7.1	Conversational Agent	12
7.1.1	Available Tools (17 total)	12
7.2	Smart Segmentation	12
7.3	Dynamic Group Scoring	13
7.4	Spam Detection	13
7.5	Merge Field AI Autocomplete	13
7.6	Engagement Heatmap and Send Performance	13

8	Analytics and Reporting	14
8.1	Platform Overview (GET /analytics/overview)	14
8.2	Campaign Detail (GET /analytics/campaigns/{id})	14
8.3	Link Analytics (GET /analytics/campaigns/{id}/links)	14
8.4	Recipient History (GET /analytics/recipients/{id})	14
8.5	CSV Exports	14
9	API Reference	14
9.1	Authentication	14
9.2	Templates	15
9.3	Campaigns	15
9.4	Analytics	15
9.5	AI	15
9.6	Recipients and Groups	16
9.7	Tracking (Public, No Auth)	16
10	Frontend Specifications	16
10.1	Design System	16
10.2	Campaign Creation Wizard	16
10.3	Template Workbench	16
10.4	Analytics Dashboards	17
10.5	AI Assistant Interface	17
11	Infrastructure and Deployment	17
11.1	Configuration Management	17
11.2	Celery and Redis	17
11.3	Kafka Event Bus	17
11.4	Application Lifecycle	18
12	Testing	18
12.1	Test Suite	18
12.2	Test Infrastructure	18
13	Appendix: Environment Variables	18
Part 2	- Detailed Design Specifications	20
14	Error Handling and Resilience	20
14.1	Graceful Degradation Strategy	20
14.2	Dispatch Error Recovery	20
14.3	AI Error Handling	20
15	Kafka Event Bus Architecture	21
15.1	Design Goals	21
15.2	Topics	21
15.3	Message Schema	21
15.4	KafkaManager Implementation	21
16	Template Engine and Versioning	22

16.1	Template Rendering Pipeline	22
16.2	Version History	22
16.3	Visual Editor Architecture	22
17	Concurrency and Async Model	22
17.1	Backend Async Architecture	22
17.2	Queue Job Claiming	23
17.3	Database Index Strategy	23
18	Detailed Sequence Flows	23
18.1	Campaign Creation and Dispatch	23
18.2	AI Agent Conversation Flow	24
18.3	Engagement Tracking Event Flow	25
19	Non-Functional Requirements	25
19.1	Performance	25
19.2	Scalability	25
19.3	Security	26
19.4	Maintainability	26
19.5	Observability	26
20	Glossary of MongoDB Collections	26

1 Introduction

1.1 Purpose

This document provides a comprehensive technical specification for the **Central Communication and Intelligent Reminder Platform (CCIRP)**. It describes the system architecture, design decisions, data models, algorithms, API contracts, AI integration, and testing strategy in sufficient detail for evaluation, maintenance, and future extension.

1.2 Scope

CCIRP is a full-stack, multi-channel communication management system supporting **Email**, **SMS**, and **WhatsApp** campaign dispatch. The platform features:

- An async Python backend (FastAPI) with MongoDB persistence.
- A modern Next.js (React 18) frontend with Tailwind CSS.
- AI-powered features: conversational assistant, smart audience segmentation, spam detection, and merge-field autocomplete - all powered by Google Gemini.
- A priority-queue-based dispatch pipeline with engagement-weighted scoring.
- Real-time open/click/unsubscribe tracking with HMAC-signed tokens.
- Comprehensive analytics with CSV exports.

1.3 Definitions and Acronyms

Term	Definition
CCIRP	Central Communication and Intelligent Reminder Platform
SSE	Server-Sent Events - streaming HTTP protocol used by the AI chat endpoint
JWT	JSON Web Token - used for stateless authentication
HMAC	Hash-based Message Authentication Code - used for tracking token integrity
Merge Field	A <code>{{placeholder}}</code> in a template body replaced at dispatch time
Dynamic Group	An audience segment resolved at send time by ranking recipients on live engagement scores
Priority Score	A composite 0–100 score determining dispatch order for each recipient

2 System Architecture

2.1 High-Level Overview

CCIRP follows a **client–server** architecture. The frontend is a single-page application communicating exclusively via RESTful JSON APIs (and one SSE streaming endpoint) with the backend.

Layer	Technology
Frontend	Next.js 14 (App Router), React 18, TypeScript, Tailwind CSS
Backend API	FastAPI (Python 3.11+), async/await throughout
Database	MongoDB 7+ via Motor (async driver), <code>tz_aware=True</code>
Authentication	JWT (HS256), bcrypt password hashing, OAuth2PasswordBearer
Email Delivery	FastAPI-Mail (SMTP) + Resend HTTP API (selectable)
SMS / WhatsApp	Twilio REST API
AI Engine	Google Gemini (<code>gemini-2.5-flash</code> / <code>gemini-3.1-flash-lite-preview</code>)
Background Jobs	Celery + Redis (graceful fallback to FastAPI BackgroundTasks)
Event Bus	Apache Kafka via <code>confluent-kafka</code> (optional, gated)
Embeddings	Sentence-BERT (<code>all-MiniLM-L6-v2</code>) for semantic tag similarity
Testing	pytest + pytest-asyncio (strict mode)

2.2 Backend Module Layout

The backend is organized into domain-driven modules under `src/`:

```
src/
+-- main.py                # FastAPI app, lifespan, CORS, router
    registration
+-- config.py              # pydantic-settings, all env vars
+-- database.py            # Motor client (tz_aware=True)
+-- kafka_utils.py         # KafkaManager, topic constants
+-- events.py              # Kafka event publishers (fail-safe)
+-- ai/                    # AI agent, tools, spam detector, merge-
    field fill
+-- auth/                  # JWT, bcrypt, OAuth2
+-- communication/         # Campaign engine, dispatch, tracking,
    email/SMS/WA
+-- templates/             # Template CRUD, preview, test-send,
    versioning
+-- recipients/            # Recipient CRUD, engagement stats model
+-- groups/                # Static groups, dynamic groups, AI
    segmentation
+-- analytics/             # Overview, campaign detail, links,
    exports
+-- reminders/             # Reminder scheduling (Celery)
+-- utils/                 # Celery task definitions
```

2.3 Frontend Module Layout

The frontend uses the Next.js App Router with file-system routing:

```
app/
+-- layout.tsx             # Root layout, AuthProvider
+-- page.tsx               # Redirects to /dashboard or /login
+-- login/ register/      # Public auth pages
+-- dashboard/
|   +-- page.tsx           # Sender analytics overview
|   +-- campaigns/[id]/    # Campaign analytics detail
```

```

+-- campaigns/
|   +-- page.tsx           # Campaign list with retry
|   +-- new/page.tsx       # 5-step campaign wizard with AI fill
+-- templates/            # Template library + visual editor
+-- recipients/           # Recipient list + profile pages
+-- groups/page.tsx       # Static and dynamic group management
+-- ai/page.tsx           # AI assistant chat interface
+-- settings/page.tsx     # User settings
components/
+-- layout/DashboardLayout # Collapsible sidebar + header
+-- templates/            # Drag-and-drop canvas editor
    components
libs/
+-- api.ts                # Centralized API client with auth
    interceptor
+-- search.ts             # Regex-aware search parsing
+-- useQueryParamState.ts # URL query param state hook

```

2.4 Data Flow

1. **Campaign Creation:** User completes 5-step wizard → POST `/campaigns/` → spam check → queue preparation → priority scoring → dispatch via Celery or BackgroundTask.
2. **Message Delivery:** For each queued job (ordered by priority), the system renders the template with merge data, injects tracking (pixel + click rewriting + unsubscribe footer), and sends via the appropriate channel provider.
3. **Engagement Tracking:** Recipient interactions hit public tracking endpoints (`/track/open`, `/track/click`, `/track/unsubscribe`) which verify HMAC tokens, record events, and update per-recipient engagement scores.
4. **Analytics:** Aggregation pipelines over `email_events` and `campaign_recipient_stats` collections produce KPIs, trends, and heatmaps.

3 Authentication and Security

3.1 JWT Token Flow

- **Access Token:** HS256-signed, 24-hour TTL. Payload contains `sub` (user email) and `exp`.
- **Refresh Token:** HS256-signed, 7-day TTL. Used via POST `/auth/refresh` to obtain a new access token without re-authentication.
- Password hashing uses `passlib` with the `bcrypt` scheme.

3.2 Request Authorization

All protected endpoints use the `get_current_active_user` dependency, which:

1. Extracts the Bearer token via `OAuth2PasswordBearer`.
2. Decodes and validates the JWT (signature + expiry).
3. Looks up the user in MongoDB by email.

4. Checks the `is_active` flag.

3.3 Tracking Token Security

Engagement tracking URLs embed HMAC-SHA256-signed tokens containing the campaign ID, recipient email, owner user ID, channel, and expiry timestamp (30-day TTL). Tokens are base64url-encoded and verified on every tracking request.

3.4 Frontend Auth

The React frontend wraps the application in an `AuthProvider` context. The `fetchAPI` wrapper in `libs/api.ts` auto-injects the `Authorization: Bearer` header and intercepts 401 responses to attempt a background token refresh before failing.

4 Data Model

4.1 MongoDB Collections

4.1.1 users

Stores registered platform users.

Field	Description
<code>email</code>	Unique user email (login identifier)
<code>hashed_password</code>	bcrypt hash
<code>full_name</code>	Display name
<code>is_active</code>	Account active flag

4.1.2 recipients

Contacts managed by each user, with rich engagement tracking.

Field	Description
<code>user_id</code>	Owner user reference
<code>email, phone</code>	Contact channels
<code>first_name, last_name</code>	Name fields
<code>tags</code>	Array of string tags for segmentation
<code>consent_flags</code>	<code>{email, sms, whatsapp}</code> booleans
<code>status</code>	<code>active</code> or <code>unsubscribed</code>
<code>engagement</code>	Nested object (see below)

Engagement Stats Sub-document:

Field	Description
open_count_total, click_count_total	Lifetime counters
bounce_count, delivery_failure_count	Delivery failure counters
tag_scores	Dict[tag_key, int] - weighted engagement per tag (opens=+2, clicks=+4)
tag_interaction_counts	Dict[tag_key, int] - unique interaction count per tag
unique_open_campaigns, unique_click_campaigns	Campaign ID arrays
last_open_at, last_click_at, last_bounced_at, unsubscribed_at	Timestamps

4.1.3 campaigns

Field	Description
name, subject	Campaign identity
template_id	Reference to the template used
channels	Array: [email, sms, whatsapp]
tags	User-defined campaign tags
recipients	Final deduplicated email list
dynamic_groups	Array of {tag, top_k, min_interactions}
merge_data	Dict[field, value] for template rendering
status	draft → scheduled queued → dispatching → sent partially_sent failed
queue_summary	Aggregated queue state (total, completed, failed, by priority level)
delivery_summary	Total/successful/failed delivery attempts
spam_score, error_message	Spam gate results

4.1.4 templates

Field	Description
name, category, channel	Template metadata
subject	Email subject line
body_html	Full HTML (email) or plain text (SMS/WhatsApp)
design_json	Serialized drag-and-drop editor state
version	Auto-incremented on each update
is_common	Shared template flag

4.1.5 campaign_dispatch_queue

Per-recipient dispatch jobs with priority ordering.

Field	Description
campaign_id, recipient_email	Unique compound key
priority_score	Composite score (0–100)
priority_level	critical (≥ 70) / high (≥ 50) / medium (≥ 28) / low
status	queued $\rightarrow$ processing $\rightarrow$ completed failed cancelled
delivery_outcome	sent, partially_sent, or failed
channel_results	Per-channel success/failure details
attempts	Retry counter

4.1.6 Other Collections

- `template_history` - archived versions on every template update.
- `campaign_recipient_stats` - per-recipient per-campaign delivery and engagement stats.
- `email_events` - raw event log (delivered, open, click, unsubscribe) with timestamps.
- `tracking_uniques` - deduplication keys for unique open/click counting.
- `groups` - static audience groups with recipient ID and email arrays.
- `dynamic_group_preferences` - saved dynamic group configurations.
- `tag_embeddings` - cached Sentence-BERT vectors for semantic similarity.
- `ai_conversations` - chat history for the AI assistant.

5 Campaign Dispatch Pipeline

5.1 Overview

The dispatch pipeline is a multi-stage, priority-queue-driven system that ensures high-value recipients receive messages first while gracefully handling failures and retries.

5.2 Campaign Lifecycle

1. **Creation:** User submits campaign via the 5-step wizard. Static group IDs are resolved to emails; dynamic groups are stored as tag/top_k/min_interactions requests.
2. **Spam Gate:** Before queuing, the system calls the AI spam detector. If the score ≥ 0.7 , the campaign is immediately set to **failed** with an error message.
3. **Queue Preparation:** For each recipient, the system computes a priority score, creates a queue job in `campaign_dispatch_queue`, and updates the campaign status to **queued**.
4. **Dispatch:** The scheduler processes jobs in priority order (**critical** $\rightarrow$ **high** $\rightarrow$ **medium** $\rightarrow$ **low**), each level with a configurable batch size.
5. **Completion:** When all jobs are terminal, the campaign status is set to **sent**, **partially_sent**, or **failed**.

5.3 Priority Scoring Algorithm

Each recipient receives a composite score $S \in [0, 100]$ computed as:

$$S = S_{\text{open}} + S_{\text{click}} + S_{\text{relationship}} + S_{\text{tag_affinity}} + S_{\text{direct_tag}} + S_{\text{recency}} + S_{\text{reliability}} + S_{\text{status}} + P_{\text{consent}} + S_{\text{channel}} \quad (1)$$

Component	Max Points	Formula
S_{open}	16	$\min\left(\frac{\ln(1+\text{open_count})}{\ln(21)}, 1\right) \times 16$
S_{click}	22	$\min\left(\frac{\ln(1+\text{click_count}+2 \times \text{unique_click_campaigns})}{\ln(19)}, 1\right) \times 22$
$S_{\text{relationship}}$	8	$\min\left(\frac{\ln(1+\text{touchpoints})}{\ln(13)}, 1\right) \times 8$
$S_{\text{tag_affinity}}$	16	$\min\left(\frac{\sum_{t \in \text{tags}} \text{tag_score}(t)}{6}, 1\right) \times 16$
$S_{\text{direct_tag}}$	8	$\min\left(\frac{ \text{recipient_tags} \cap \text{campaign_tags} }{3}, 1\right) \times 8$
S_{recency}	16	Tiered: $\leq 3\text{d} \rightarrow 16$, $\leq 7\text{d} \rightarrow 12$, $\leq 14\text{d} \rightarrow 9$, $\leq 30\text{d} \rightarrow 6$, $\leq 90\text{d} \rightarrow 3$, else 0
$S_{\text{reliability}}$	6	$\frac{\text{delivered}}{\text{delivered} + \text{failed}} \times 6$ (default 3 if no history)
S_{status}	5	+5 if active , -20 otherwise
P_{consent}	0	-15 if email consent revoked; -6 per SMS/WhatsApp revoked
S_{channel}	5	+5 if phone present (for non-email), else -6

Priority Levels:

- **critical**: $S \geq 70$ (batch size: 25)
- **high**: $S \geq 50$ (batch size: 20)
- **medium**: $S \geq 28$ (batch size: 15)
- **low**: $S < 28$ (batch size: 10)

5.4 Scheduler

The `run_campaign_scheduler` coroutine runs inside the FastAPI process on a configurable interval (default 15s). Each tick:

1. Prepares queues for campaigns that are **queued/scheduled** but not yet prepared.
2. Requeues stale **processing** jobs (stuck > 300s).
3. Processes one batch of jobs per priority level.

If Celery+Redis are available, `ccirp.scheduler_tick` also runs as a Celery Beat task.

5.5 Retry Mechanism

POST `/campaigns/{id}/retry` is only callable when status is **failed** or **partially_sent**. It:

- Resets all **failed/cancelled** queue entries to **queued** (zeroes attempts).
- Sets campaign status to **queued** (fully failed) or **dispatching** (partial - some already completed).

- Re-triggers dispatch via Celery or BackgroundTask.

5.6 Multi-Channel Rendering

For each recipient-channel pair:

1. **Template Rendering:** All `{{field}}` placeholders are replaced using campaign merge data plus auto-injected fields (`name`, `email`, `first_name`).
2. **Email:** HTML body receives click-tracking link rewriting, an unsubscribe footer, and a 1×1 tracking pixel before `</body>`.
3. **SMS/WhatsApp:** HTML is converted to plain text via regex-based tag stripping; URLs are rewritten for click tracking.

6 Engagement Tracking

6.1 Tracking Endpoints (Public, No Auth)

Method	Path	Behavior
GET	<code>/track/open/{token}.png</code>	Returns 1×1 transparent PNG; records open event (email only)
GET	<code>/track/click/{token}?u={url}</code>	Records click event; 302 redirects to original URL
GET	<code>/track/unsubscribe/{token}</code>	Sets recipient status to <code>unsubscribed</code> ; revokes all consent flags; returns styled HTML confirmation

6.2 Event Recording

Each event is stored in the `email_events` collection with fields: `event_type`, `campaign_id`, `recipient_email`, `channel`, `is_unique`, `link_url` (for clicks), `ip`, `user_agent`, and `ts`.

Unique deduplication uses a compound key stored in `tracking_uniques`: `type:campaign:recipient_email`.

6.3 Engagement Score Updates

On each unique event, the recipient's engagement document is updated:

- **Unique Open:** `tag_scores[tag] += 2`, `tag_interaction_counts[tag] += 1` for each campaign tag.
- **Unique Click:** `tag_scores[tag] += 4`, `tag_interaction_counts[tag] += 1` for each campaign tag. Clicked domain is added to `clicked_domains`.
- **Delivery Failure:** `bounce_count` and `delivery_failure_count` are incremented; `last_bounced_at` is stamped.
- **Unsubscribe:** All `consent_flags` set to `false`; `unsubscribed_at` timestamped. Unsubscribe is **global** - revokes all channels simultaneously.

7 AI-Powered Features

7.1 Conversational Agent

Endpoint: POST /ai/chat (streaming SSE, text/event-stream).

Model: Google Gemini (gemini-3.1-flash-lite-preview), configurable via MODEL_NAME.

Architecture: The agent uses a tool-calling loop with up to 6 iterations (MAX_TOOL_ITERATIONS).

Two parallel content lists are maintained:

- **api_contents:** Raw proto Content objects preserving the thought_signature field required by Gemini thinking models.
- **db_contents:** Plain dicts safe for MongoDB storage.

A nudge mechanism fires if all iterations produced tool calls but no visible text, recovering the response with an explicit prompt.

7.1.1 Available Tools (17 total)

Tool	Description
search_recipients	Regex search across name, email, and tags
get_recipient_detail	Full engagement stats, consent flags, tag scores
list_campaigns	Campaigns sorted by newest, with delivery totals
get_campaign_detail	Full analytics including queue summary
list_templates	Available templates with category and version
get_template_detail	Full template content including body_html
create_template	Create production-ready HTML/text templates
update_template	Update existing templates with version increment
list_static_groups	All static groups with recipient counts
list_dynamic_preferences	Saved dynamic group configurations
preview_dynamic_group	Live top-K recipients ranked by tag score
preview_ai_segmentation	Semantic similarity-based audience discovery
create_static_group	Persist a new static group
save_dynamic_preference	Save/update a dynamic group preference
get_analytics_overview	Platform-wide KPIs and top tags
get_engagement_heatmap	Open/click counts by hour × day-of-week (UTC)
get_campaign_send_performance	Per-campaign send timing vs. engagement metrics

7.2 Smart Segmentation

Endpoint: POST /groups/segmentation

This feature uses **Sentence-BERT** (all-MiniLM-L6-v2) to compute semantic similarity between a target tag and all existing tags in the system. The algorithm:

1. **Embedding:** Each tag string is encoded into a 384-dimensional vector. Embeddings are cached in the tag_embeddings collection.
2. **Similarity:** Cosine similarity is computed between the target tag embedding and every candidate group's tag embedding.
3. **Filtering:** Groups with similarity below the threshold (default 0.15) are discarded.
4. **Normalization:** Remaining scores are min-max normalized to [0, 1].

5. **Weighting:** Either proportional or softmax (temperature-controlled) weighting determines each group's allocation share.
6. **Allocation:** A largest-remainder method distributes `max_output_size` slots across groups, ensuring each matched group gets at least one slot.
7. **Selection:** Within each group, recipients are ranked by their dynamic engagement score and selected up to the allocation count.

This allows the system to, for example, match the tag “machine-learning” with recipients tagged “artificial-intelligence”, “deep-learning”, or “data-science” - significantly broadening campaign reach without manual audience curation.

7.3 Dynamic Group Scoring

Dynamic groups rank recipients by a composite score per tag:

$$D = S_{\text{tag}} + S_{\text{interaction}} + S_{\text{open}} + S_{\text{click}} + S_{\text{relationship}} + S_{\text{recency}} + S_{\text{reliability}} + S_{\text{status}} + S_{\text{consent}} \quad (2)$$

where S_{tag} contributes up to 38 points (log-scaled from historical `tag_scores`) and S_{click} up to 18 points, emphasizing click intent as the strongest engagement signal.

7.4 Spam Detection

Endpoint: `POST /campaigns/check-spam` (manual) + automatic pre-dispatch gate.

A standalone Gemini prompt evaluates the campaign subject and body content as a channel-specific spam filter. The model returns:

- **is_spam:** boolean (true if score ≥ 0.7)
- **score:** float $\in [0.0, 1.0]$
- **reason:** qualitative explanation string

When integrated into the dispatch pipeline, spam-flagged campaigns are immediately set to **failed** with the spam reason as the error message, preventing any messages from being sent.

7.5 Merge Field AI Autocomplete

Endpoint: `POST /ai/fill-merge-fields`

A non-streaming Gemini call that analyzes campaign context (name, subject, free-text intent description) to generate contextually appropriate values for custom merge fields. The model returns a JSON object mapping each requested field name to a suggested value. Auto-injected fields like `{{name}}` and `{{email}}` are excluded - only user-defined placeholders (e.g., `{{promo_code}}`, `{{deadline}}`) are filled.

7.6 Engagement Heatmap and Send Performance

Two analytics tools available to the AI agent (and internally):

- **Engagement Heatmap:** Aggregates all unique open and click events by hour-of-day (0–23 UTC) and day-of-week, producing ranked “best send windows” for time optimization recommendations.
- **Send Performance:** Per-campaign metrics including send time, open rate, click rate, and average time-to-first-open in hours - enabling correlation of dispatch timing with engagement outcomes.

8 Analytics and Reporting

8.1 Platform Overview (GET /analytics/overview)

- Total campaigns, total messages sent, unique opens/clicks, open/click/bounce/unsubscribe rates.
- 30-day daily trend: opened, clicked, unsubscribed counts per day.
- Top 5 engaged tags ranked by total open+click events.
- Recent campaign performance summaries.

8.2 Campaign Detail (GET /analytics/campaigns/{id})

- Campaign name, channels, and channel-aware metrics.
- 72-hour hourly timeline (opens/clicks).
- Per-recipient activity with status labels (Clicked > Opened > Failed > Delivered).
- Delivery failure count and error messages.

8.3 Link Analytics (GET /analytics/campaigns/{id}/links)

Per-URL click totals and unique click counts, ranked by popularity.

8.4 Recipient History (GET /analytics/recipients/{id})

Full engagement history for one recipient: campaigns participated in, per-campaign delivery/open/click status, and resolved campaign names.

8.5 CSV Exports

Three export endpoints return `text/csv` responses:

- /analytics/campaigns/{id}/export - campaign detail CSV.
- /analytics/campaigns/{id}/links/export - link analytics CSV.
- /analytics/overview/export - all campaigns summary CSV.

9 API Reference

9.1 Authentication

Method	Path	Description
POST	/auth/register	Register a new user account
POST	/auth/login	Authenticate; returns access + refresh tokens
POST	/auth/refresh	Exchange refresh token for new access token
GET	/auth/me	Return current authenticated user profile

9.2 Templates

Method	Path	Description
GET	/templates/	List all templates (user-owned + common)
POST	/templates/	Create a new template
PUT	/templates/{id}	Update template (archives previous version)
DELETE	/templates/{id}	Delete a template
POST	/templates/preview	Server-side render with sample merge data
POST	/templates/{id}/test-send	Dispatch a real test email

9.3 Campaigns

Method	Path	Description
GET	/campaigns/	List all campaigns (paginated)
POST	/campaigns/	Create campaign + enqueue for dispatch
GET	/campaigns/{id}	Get campaign details
GET	/campaigns/{id}/analytics	Full campaign analytics
POST	/campaigns/{id}/retry	Retry failed/partially_sent campaign
POST	/campaigns/check-spam	Manual spam pre-check

9.4 Analytics

Method	Path	Description
GET	/analytics/overview	Platform-wide stats + 30-day trend
GET	/analytics/overview/export	CSV export of all campaigns
GET	/analytics/campaigns/{id}	Campaign detail analytics
GET	/analytics/campaigns/{id}/links	Per-URL click statistics
GET	/analytics/campaigns/{id}/export	Campaign CSV export
GET	/analytics/campaigns/{id}/links/export	Link analytics CSV
GET	/analytics/recipients/{id}	Recipient engagement history

9.5 AI

Method	Path	Description
POST	/ai/chat	Streaming SSE conversational agent
POST	/ai/merge-fields	AI merge field value suggestions
GET	/ai/conversations	List conversation history
GET	/ai/conversations/{id}	Get full conversation with messages
DELETE	/ai/conversations/{id}	Delete a conversation

9.6 Recipients and Groups

Method	Path	Description
GET/POST	/recipients/	List / create recipients
GET/PUT/DELETE	/recipients/{id}	CRUD operations
GET/POST	/groups/	List / create static groups
GET	/groups/dynamic-preferences	List saved dynamic configs
POST	/groups/resolve-dynamic	Resolve dynamic audience in real-time
POST	/groups/segmentation	AI semantic segmentation

9.7 Tracking (Public, No Auth)

Method	Path	Description
GET	/track/open/{token}.png	Open pixel (1×1 PNG)
GET	/track/click/{token}	Click redirect
GET	/track/unsubscribe/{token}	Global opt-out

10 Frontend Specifications

10.1 Design System

- **Framework:** Tailwind CSS with HSL root variables and a dark-first theme.
- **Animations:** `animate-fade-in`, `animate-fade-up`, shimmer loading skeletons.
- **Glassmorphism:** `backdrop-blur` and translucent `bg-card/90` surfaces.
- **Layout:** Collapsible sidebar + contextual header shell (`DashboardLayout`).

10.2 Campaign Creation Wizard

A 5-step progressive form:

1. **Campaign Details:** Name, subject, channel toggles (Email/SMS/WhatsApp), scheduled dispatch time, tags.
2. **Select Template:** Grid of template cards with category pills and version badges.
3. **Merge Fields:** Auto-extracted `{{field}}` placeholders with live preview. AI Fill Assist panel calls `POST /ai/fill-merge-fields` to auto-populate contextual values.
4. **Audience:** Static groups, dynamic group preferences with top-K override and live preview, individual user selection. Phone-readiness warnings for SMS/WhatsApp.
5. **Review:** Summary cards, spam check gate (calls `POST /campaigns/check-spam`), final submit.

10.3 Template Workbench

- Visual drag-and-drop canvas with infinite dotted grid, Ctrl+Wheel zoom, Spacebar-drag pan.
- Layers panel reflecting real-time block hierarchy.

- Typography, color, and layout property inspector.
- Template version history with rollback.
- Test send modal for dispatching real emails with sample data.

10.4 Analytics Dashboards

- **Sender Overview:** KPI cards, 30-day Recharts area chart (opened/clicked/unsubscribed), top engaged segments, recent campaigns table, CSV export.
- **Campaign Detail:** KPI cards, 72-hour delivery timeline, recipient activity table, top clicked links panel, per-campaign CSV/links export.
- **Recipient Profile:** 4 KPI cards (opens/clicks/bounces/failures), activity timesamps, campaign history list.

10.5 AI Assistant Interface

Full chat interface streaming SSE events from `POST /ai/chat`. Tool invocations are displayed inline (`tool_start/tool_result` events). Conversation sidebar lists past conversations with create/delete functionality. AI responses are rendered as Markdown.

11 Infrastructure and Deployment

11.1 Configuration Management

All settings are centralized in `src/config.py` using `pydantic-settings`, reading from a `.env` file. Key configuration groups:

- **Database:** `MONGODB_URL`, `DATABASE_NAME`
- **Auth:** `SECRET_KEY`, `ALGORITHM`, token expiry durations
- **Email:** Provider selection (`smtp/resend`), SMTP credentials, Resend API key
- **SMS/WhatsApp:** Twilio credentials and sender numbers
- **Tracking:** Base URL, HMAC signing key, token TTL
- **AI:** `GOOGLE_API_KEY`
- **Queue:** Scheduler interval, batch sizes per priority level, stale timeout
- **Kafka:** Bootstrap servers, enable/disable flag

11.2 Celery and Redis

Celery tasks (`ccirp.dispatch_campaign`, `ccirp.scheduler_tick`, `ccirp.send_reminder`) use Redis as broker and result backend. The system gracefully degrades: if Celery/Redis are unavailable, `FastAPI BackgroundTasks` handle dispatch inline.

11.3 Kafka Event Bus

Optional event publishing to `ccirp.campaign.events` and `ccirp.delivery.events` topics. Gated by the `KAFKA_ENABLED` flag - when disabled, `produce_message` returns `False` immediately, preventing `librdkafka`'s background reconnect thread from flooding `stderr`.

11.4 Application Lifecycle

`src/main.py` uses FastAPI's `lifespan` context manager to:

1. Connect to MongoDB on startup.
2. Start the campaign scheduler coroutine.
3. Cancel the scheduler and close the MongoDB connection on shutdown.

12 Testing

12.1 Test Suite

The project includes 7 test modules under `tests/`:

Module	Coverage
<code>test_engagement_profiles</code>	26 async tests: engagement stats defaults, bounce rollup, unsubscribe timestamps, plain-text click tracking, link analytics, recipient history, unsubscribe rate
<code>test_priority_scoring</code>	Priority score calculation across various recipient engagement profiles
<code>test_spam_detector</code>	AI spam detection with mock Gemini responses
<code>test_celery_tasks</code>	Celery task dispatch and fallback behavior
<code>test_kafka_events</code>	Kafka event publishing with enable/disable gating
<code>test_recipients_db</code>	Recipient CRUD and engagement model integrity
<code>test_templates_db</code>	Template versioning and history archival

12.2 Test Infrastructure

- **Framework:** `pytest` with `pytest-asyncio` in strict mode.
- **Mock Pattern:** `_make_db(collections)` factory with `MagicMock(side_effect=fn)` for MongoDB collection dispatch.
- All async tests are decorated with `@pytest.mark.asyncio`.

13 Appendix: Environment Variables

Variable	Default	Notes
<code>MONGODB_URL</code>	<code>localhost:27017</code>	
<code>DATABASE_NAME</code>	<code>ccirp_db</code>	
<code>SECRET_KEY</code>	(dev key)	JWT signing
<code>ACCESS_TOKEN_EXPIRE_MINUTES</code>	1440	24 hours
<code>REFRESH_TOKEN_EXPIRE_DAYS</code>	7	
<code>EMAIL_PROVIDER</code>	<code>smtp</code>	<code>smtp</code> or <code>resend</code>
<code>SMTP_HOST/PORT/USER/PASSWORD</code>	-	Gmail requires app password
<code>MAIL_FROM</code>	-	Must match <code>SMTP_USER</code> for Gmail
<code>TWILIO_ACCOUNT_SID/AUTH_TOKEN</code>	-	For SMS/WhatsApp

Variable	Default	Notes
TRACKING_BASE_URL	localhost:8000	Base for pixel/link URLs
TRACKING_SIGNING_KEY	(dev key)	HMAC key
TRACKING_TOKEN_TTL_SECONDS	2592000	30 days
GOOGLE_API_KEY	-	Gemini API key
KAFKA_ENABLED	True	Set False if Kafka is down
CAMPAIGN_SCHEDULER_INTERVAL_SECONDS	60	
CAMPAIGN_QUEUE_BATCH_SIZE_CRITICAL	1	
CAMPAIGN_QUEUE_BATCH_SIZE_HIGH	20	
CAMPAIGN_QUEUE_BATCH_SIZE_MEDIUM	15	
CAMPAIGN_QUEUE_BATCH_SIZE_LOW	10	
CAMPAIGN_QUEUE_STALE_SECONDS	300	

Part 2 - Detailed Design Specifications

14 Error Handling and Resilience

14.1 Graceful Degradation Strategy

CCIRP is designed to operate correctly even when optional infrastructure components are unavailable:

Component	Detection	Fallback Behavior
Celery / Redis	Import or <code>.delay()</code> exception	Dispatch falls back to FastAPI <code>BackgroundTasks</code> for inline async processing
Apache Kafka	<code>KAFKA_ENABLED = False</code> or producer init failure	<code>produce_message</code> returns <code>False</code> silently; no events published
Sentence-BERT	<code>ImportError</code> on <code>sentence_transformers</code>	HTTP 503 with descriptive error; segmentation endpoint is unavailable but all other features continue
Twilio	Missing credentials	Per-channel failure; email channel continues; <code>channel_results</code> records the error
Resend API	Missing <code>RESEND_API_KEY</code>	Returns <code>(False, "RESEND_API_KEY is not configured")</code>

14.2 Dispatch Error Recovery

- **Per-Channel Isolation:** Each channel (email, SMS, WhatsApp) is attempted independently. A failure in SMS does not block email delivery for the same recipient.
- **Outcome Classification:** A queue job is marked `completed` / `partially_sent` if at least one channel succeeds, or `failed` only if all channels fail.
- **Stale Job Recovery:** The scheduler requeues any job stuck in `processing` for longer than `CAMPAIGN_QUEUE_STALE_SECONDS` (default 300s), preventing resource leaks from crashed workers.
- **Campaign-Level Retry:** The `POST /campaigns/{id}/retry` endpoint allows re-dispatching all failed/cancelled jobs while preserving completed deliveries.

14.3 AI Error Handling

- **Tool Call Failures:** If a tool function raises an exception, the error message is returned as the tool result, allowing the AI agent to self-correct or inform the user.
- **Nudge Mechanism:** If the agent exhausts all `MAX_TOOL_ITERATIONS` (6) without producing visible text, a recovery prompt is injected: *"Please provide a final text summary of what you found or did."* This prevents silent failures.
- **Spam Detection Fallback:** If the Gemini spam analysis call fails (network error, quota), the system defaults to allowing the campaign through rather than blocking

silently. The error is logged.

15 Kafka Event Bus Architecture

15.1 Design Goals

The Kafka integration provides an asynchronous event bus for decoupling campaign life-cycle events from the request-response cycle. This enables future consumers (analytics pipelines, webhooks, audit logs) without modifying core dispatch logic.

15.2 Topics

Topic	Event Types
ccirp.campaign.events	campaign_created, campaign_status_changed - published on campaign creation and every status transition
ccirp.delivery.events	delivery_success, delivery_failed - published per-recipient per-channel after each send attempt

15.3 Message Schema

Each Kafka message is a JSON-serialized dictionary:

```
{
  "event_type": "delivery_success",
  "campaign_id": "665a1b...",
  "recipient_email": "user@example.com",
  "channel": "email",
  "delivered": true,
  "error_message": null,
  "timestamp": "2026-05-06T14:22:00Z"
}
```

15.4 KafkaManager Implementation

The `KafkaManager` is a singleton that lazily initializes a `confluent_kafka.Producer` on first use. Key design decisions:

- **Gated Initialization:** If `KAFKA_ENABLED` is `False`, the producer is never created, avoiding `librdkafka`'s background reconnection thread.
- **Flush on Produce:** Each `produce_message` call is followed by a `flush(timeout=5)` to ensure delivery without requiring a separate background flusher.
- **Fail-Safe:** All publish functions catch exceptions silently - Kafka availability never blocks campaign dispatch.

16 Template Engine and Versioning

16.1 Template Rendering Pipeline

Template rendering follows a three-stage pipeline:

1. **Merge Field Extraction:** A regex scanner identifies all `{{field_name}}` patterns in the HTML body. These are presented to the user in the wizard's Step 3.
2. **Auto-Field Injection:** At dispatch time, the following fields are injected per-recipient without user input:
 - `{{name}}` - full name from recipient record
 - `{{email}}` - recipient email address
 - `{{first_name}}`, `{{last_name}}` - name components
3. **Substitution:** All merge fields are replaced using Python string replacement. Unresolved fields are left as-is (visible as raw `{{field}}` in the output), providing visual feedback rather than silent omission.

16.2 Version History

On every template update via `PUT /templates/{id}`, the system:

1. Archives the current version to `template_history` (full snapshot: `body_html`, `design_json`, `subject`, `version`).
2. Increments the `version` counter on the live document.
3. Updates the `updated_at` timestamp.

The frontend exposes a version history panel with rollback functionality, restoring both the visual editor state (`design_json`) and the raw HTML.

16.3 Visual Editor Architecture

The drag-and-drop editor is implemented as a React canvas component with:

- **Infinite grid:** Dotted pattern background with configurable spacing.
- **Zoom:** Ctrl+Wheel, with smooth CSS transform scaling.
- **Pan:** Spacebar+drag for viewport navigation.
- **Layers Panel:** Real-time tree view of all blocks, supporting reordering and selection.
- **Property Inspector:** Per-block typography, color, spacing, and layout controls.
- **Serialization:** Editor state is serialized to `design_json` (block tree with positions, styles, and content) alongside the rendered `body_html`.

17 Concurrency and Async Model

17.1 Backend Async Architecture

The entire backend is built on Python's `asyncio` event loop via FastAPI:

- **Motor:** All MongoDB operations use the async Motor driver with `tz_aware=True` to prevent timezone-related rendering bugs on the frontend.
- **HTTP Calls:** Email (Resend) and SMS/WhatsApp (Twilio) providers use `asyncio.to_thread()` to offload blocking `urllib` requests to a thread pool, preventing event loop starvation.

- **Scheduler:** The campaign scheduler runs as an `asyncio.create_task()` coroutine within the FastAPI process, using `asyncio.wait_for(stop_event.wait(), timeout=interval)` for clean shutdown.
- **SSE Streaming:** The AI chat endpoint uses an async generator that yields SSE-formatted chunks, leveraging FastAPI's `StreamingResponse`.

17.2 Queue Job Claiming

To prevent double-processing in multi-worker scenarios, queue jobs are claimed atomically using MongoDB's `find_one_and_update`:

```
await db["campaign_dispatch_queue"].find_one_and_update(
    {"priority_level": level, "status": "queued",
     "campaign_id": campaign_id},
    {"$set": {"status": "processing",
              "processing_started_at": now,
              "attempts": {"$ifNull": ["$attempts", 0]} + 1}},
    sort=[("priority_score", -1), ("created_at", 1)],
    return_document=ReturnDocument.AFTER,
)
```

This ensures that even with concurrent scheduler instances, each job is claimed exactly once.

17.3 Database Index Strategy

Critical indexes for performance:

- `campaign_dispatch_queue`: Compound index on (`priority_level`, `status`, `campaign_id`, `priority_score` DESC) for efficient batch claiming.
- `campaign_recipient_stats`: Compound index on (`campaign_id`, `recipient_email`, `channel`) for per-recipient lookups.
- `email_events`: Index on (`campaign_id`, `event_type`) and (`owner_user_id`, `ts`) for analytics aggregation.
- `dynamic_group_preferences`: Unique index on (`created_by`, `tag_key`).
- `tag_embeddings`: Unique index on (`model_name`, `text_key`).
- `tracking_uniques`: Uses `_id` as the unique key directly (e.g., `open:campaign:email:channel`).

18 Detailed Sequence Flows

18.1 Campaign Creation and Dispatch

1. User completes the 5-step wizard and clicks "Send Campaign".
2. Frontend calls `POST /campaigns/check-spam` with subject and template ID.
3. Backend loads the template, converts HTML to plain text, and sends both to Gemini for spam analysis.
4. If `is_spam = true`, the frontend blocks submission and displays the reason.
5. If clean, frontend calls `POST /campaigns/` with the full campaign payload.

6. Backend resolves static group IDs to emails, deduplicates, and stores the campaign as `queued`.
7. Backend attempts `dispatch_campaign_task.delay(campaign_id)` via Celery.
8. If Celery is unavailable, falls back to `BackgroundTasks.add_task(prepare_campaign_priority...)`.
9. The dispatch function calls `enqueue_campaign_recipients`:
 - a. Resolves dynamic group audiences (tag scoring + top-K selection).
 - b. Merges dynamic emails with static recipient list (deduplication).
 - c. Runs the spam gate on the campaign content.
 - d. For each recipient, computes the priority score and inserts a queue job.
 - e. Stores `queue_summary` on the campaign document.
10. The scheduler loop picks up queued jobs by priority level, processes each through `_process_priority_queue_job`:
 - a. Loads the campaign and template.
 - b. Renders the template with merge data and recipient-specific auto-fields.
 - c. For each channel: injects tracking, sends via the provider, records delivery event.
 - d. Publishes a Kafka delivery event (if enabled).
 - e. Marks the queue job as terminal.
11. After all jobs complete, `_sync_campaign_queue_state` updates the campaign status to `sent`, `partially_sent`, or `failed`.

18.2 AI Agent Conversation Flow

1. User sends a message via the chat interface.
2. Frontend calls `POST /ai/chat` with `conversation_id` and `message`.
3. Backend loads conversation history from `ai_conversations`, appends the user message.
4. The agent calls `model.generate_content(api_contents, stream=True)` with the system prompt and all 17 tool declarations.
5. The response is streamed chunk-by-chunk via SSE:
 - `data: {"type": "text", "content": "..."} for visible text.`
 - `data: {"type": "tool_start", "name": "...", "args": {...}}` when the model invokes a tool.
 - `data: {"type": "tool_result", "name": "...", "result": {...}}` after tool execution.
6. If the model requests a tool call, the backend:
 - a. Extracts the function name and arguments.
 - b. Dispatches to the corresponding Python function in `src/ai/tools.py`.

- c. Appends the tool response to `api_contents` (with thought signature) and `db_contents` (without).
 - d. Re-calls `generate_content` for the model to process the tool result.
7. This loop continues for up to 6 iterations. If the model produces only tool calls without text, the nudge mechanism appends a recovery prompt.
8. The complete conversation (user message + assistant response + tool calls/results) is saved to MongoDB.

18.3 Engagement Tracking Event Flow

1. Recipient opens an email; their email client loads the tracking pixel URL.
2. `GET /track/open/{token}.png` is received by the backend.
3. The HMAC-SHA256 token is verified (signature + expiry check).
4. The payload is decoded: `campaign_id`, `recipient_email`, `owner_user_id`, `channel`.
5. `record_engagement_event` is called:
 - a. A uniqueness check is performed against `tracking_uniques`.
 - b. A raw event is inserted into `email_events`.
 - c. `campaign_recipient_stats` is updated (counters, timestamps).
 - d. The recipient's engagement document is updated (tag scores, interaction counts).
6. A 1×1 transparent PNG is returned to the email client.

19 Non-Functional Requirements

19.1 Performance

- **Batch Processing:** Priority queue dispatch processes configurable batch sizes (10–25 jobs per priority level per scheduler tick) to balance throughput with resource consumption.
- **Embedding Cache:** Sentence-BERT embeddings are computed once and cached in MongoDB, eliminating redundant model inference for repeated tags.
- **Aggregation Pipelines:** Analytics use MongoDB aggregation pipelines executed server-side, minimizing data transfer for operations like trend computation and tag ranking.
- **Streaming AI:** SSE streaming ensures the user sees AI responses incrementally rather than waiting for full generation.

19.2 Scalability

- **Horizontal Scaling:** Celery workers can be scaled independently. MongoDB's atomic `find_one_and_update` ensures safe concurrent job claiming.
- **Event Decoupling:** Kafka enables adding downstream consumers (e.g., real-time dashboards, webhook dispatchers) without modifying core dispatch logic.
- **Stateless API:** All session state is in JWT tokens; the FastAPI process itself is stateless and can be replicated behind a load balancer.

19.3 Security

- **Authentication:** JWT with bcrypt password hashing; configurable token lifetimes.
- **Authorization:** Every data-modifying endpoint verifies `created_by` matches the authenticated user, preventing cross-tenant data access.
- **Tracking Integrity:** HMAC-SHA256 tokens prevent URL tampering; expired tokens are rejected.
- **CORS:** Restricted to the configured frontend URL only.
- **Input Validation:** Pydantic models enforce schema validation on all API inputs.
- **Consent Enforcement:** Recipients with revoked consent flags receive a negative priority penalty (−15 for email, −6 per SMS/WhatsApp), naturally deprioritizing them.

19.4 Maintainability

- **Domain-Driven Modules:** Each feature area (AI, communication, groups, analytics, etc.) is a self-contained module with its own router, service, schemas, and models.
- **Centralized Configuration:** Single `pydantic-settings` class with environment variable overrides.
- **Comprehensive Testing:** 7 test modules covering engagement profiles, priority scoring, spam detection, Celery tasks, Kafka events, and CRUD operations.
- **Type Safety:** Full TypeScript on the frontend; Pydantic models on the backend.

19.5 Observability

- **Event Log:** Every delivery attempt, open, click, and unsubscribe is recorded in `email_events` with timestamps, enabling full audit trails.
- **Queue State:** The `campaign_dispatch_queue` collection provides real-time visibility into dispatch progress, with `queue_summary` aggregated on the campaign document.
- **Kafka Events:** When enabled, all lifecycle and delivery events are published for external monitoring and alerting systems.

20 Glossary of MongoDB Collections

Collection	Purpose
<code>users</code>	Registered platform users with auth credentials
<code>recipients</code>	Contact records with engagement stats, tags, and consent flags
<code>campaigns</code>	Campaign definitions with audience, merge data, and lifecycle status
<code>templates</code>	Email/SMS/WhatsApp content templates with HTML and editor state
<code>template_history</code>	Archived template versions for rollback

Collection	Purpose
groups	Static audience groups (named collections of recipient IDs)
dynamic_group_preferences	Saved dynamic group configurations (tag, top_k, min_interactions)
campaign_dispatch_queue	Per-recipient dispatch jobs with priority scores and delivery outcomes
campaign_recipient_stats	Per-recipient per-campaign delivery and engagement metrics
email_events	Raw event log (delivered, open, click, unsubscribe)
tracking_uniques	Deduplication keys for unique open/click counting
tag_embeddings	Cached Sentence-BERT vectors indexed by tag text
ai_conversations	AI assistant conversation history with tool call records
reminders	Scheduled reminder entries (Celery-driven)